Conversación sobre la tarea de Java

Versión ampliada — Manejo de ficheros en Java · DAM · Curso 2026-2027

1. Inicio de la conversación

El usuario solicitó ayuda para resolver en Java una tarea evaluativa de la Unidad Didáctica 1, centrada en el manejo de ficheros. La petición inicial fue: «resuelve este ejercicio para java». El documento aportado corresponde a una actividad de DAM en la que se pide aplicar diferentes mecanismos de entrada y salida de datos.

2. Objetivo general de la actividad

La tarea trabaja lectura y escritura de caracteres, lectura y escritura mediante buffers, lectura de datos binarios, comprobación de cabeceras y acceso aleatorio a registros de longitud fija.

Los ejercicios deben ser independientes y utilizar las clases indicadas en cada apartado. También se piden rutas relativas, gestión de recursos y errores, compilación, ejecución y pruebas.

3. Ejercicio 1 — FileReader y FileWriter

Se debe leer «entrada.txt» carácter a carácter con FileReader y generar un fichero de salida con FileWriter. Las letras se convierten a mayúsculas, los dígitos del 0 al 9 se sustituyen por «#» y el resto de caracteres, incluidos espacios y saltos de línea, se conserva.

El programa debe leer hasta EOF, cerrar los recursos correctamente y gestionar posibles errores de entrada/salida.

4. Ejercicio 2 — BufferedReader y BufferedWriter

A partir de «accesos.log», cuyas líneas tienen el formato «fecha;usuario;resultado», se debe crear «errores.log» conservando únicamente las líneas cuyo resultado sea ERROR. Al final se añade el número total de errores.

Se solicita BufferedReader.readLine(), BufferedWriter, soporte para un número arbitrario de líneas y cierre correcto de recursos.

5. Ejercicio 3 — comprobación de una cabecera ZIP

El programa recibe la ruta del fichero como argumento y lee únicamente los cuatro primeros bytes mediante InputStream/FileInputStream. La firma esperada es 80, 75, 3 y 4.

Si coincide, se muestra «La cabecera es compatible con un fichero ZIP.» Si no coincide, se informa de que la firma no es la esperada. Si hay menos de cuatro bytes, debe comunicarse explícitamente. La cabecera compatible no demuestra por sí sola que el fichero completo sea un ZIP válido.

6. Ejercicio 4 — inventario con RandomAccessFile

El inventario binario se almacena en «inventario.dat» mediante registros de longitud fija. Los campos indicados por la tarea son:

Campo	Tipo	Longitud
Id	int	—
Código	String	8 caracteres
Nombre	String	20 caracteres
Categoría	String	12 caracteres
Stock	int	—

Precio	double	—

6.1. Crear inventario.dat

La parte 4.a pide crear el fichero con RandomAccessFile en modo lectura/escritura, cargando los datos iniciales proporcionados. Los IDs comienzan en 1 y los campos String tienen longitud fija, por lo que deben rellenarse o truncarse según sea necesario. El programa debe calcular y mostrar el número de registros y el tamaño del registro.

6.2. ActualizarStock

«ActualizarStock» recibe el ID y el nuevo stock, por ejemplo «java ActualizarStock 3 17». Debe localizar el registro directamente con seek(), comprobar que existe y que el stock no es negativo, leer el nombre y el stock anterior y modificar únicamente el campo stock.

La salida debe mostrar el nombre, el stock anterior, el nuevo stock y la variación.

6.3. ListarCategoria

«ListarCategoria» recibe una categoría por argumentos, recorre los registros existentes y muestra todos los campos de los registros que coinciden. La comparación es independiente de mayúsculas y minúsculas. No se deben hardcodear categorías y se debe indicar el total encontrado o un mensaje claro si no hay coincidencias.

7. Archivos Java separados

• TransformarFichero.java — transformación de entrada.txt.

• FiltrarErrores.java — filtrado de accesos.log.

• ComprobarZip.java — comprobación de la firma ZIP.

• Equipo.java — modelo de un registro de inventario.

• DatosInventario.java — datos auxiliares del inventario.

• CrearInventario.java — creación de inventario.dat.

• ActualizarStock.java — modificación directa del stock.

• ListarCategoria.java — listado por categoría.

8. Tamaño del registro de inventario

En la solución preparada se representan los String mediante writeChar/readChar, por lo que cada carácter ocupa 2 bytes. El tamaño calculado es: 4 + (8×2) + (20×2) + (12×2) + 4 + 8 = 96 bytes.

Con este diseño, el registro correspondiente a un ID se obtiene mediante (id - 1) × 96. El stock comienza en el desplazamiento 84 dentro del registro: 4 + 16 + 40 + 24 = 84.

9. Extracción de los datos de las tablas

Cuando el usuario pidió «saca los datos de las tabla del documento», se comprobó el contenido disponible. La tabla del inventario proporciona la estructura de los campos, pero el documento indica que los datos iniciales se encuentran en un fichero externo llamado «ArraysInventario.txt».

Por tanto, no se añadieron valores inventados de código, nombre, categoría, stock o precio. Para extraer exactamente las filas iniciales del inventario sería necesario disponer del fichero ArraysInventario.txt si esos valores no están incluidos en el documento.

10. Proceso de trabajo y uso de IA

La tarea incluye una autoevaluación en la que se pide documentar el uso de IA: herramienta utilizada, finalidad, prompts empleados, decisiones aceptadas, modificadas o rechazadas, pruebas realizadas, errores detectados y reflexión sobre la comprensión del código.

11. Pruebas que contempla la tarea

• Comprobar el funcionamiento normal de cada programa.

• Probar casos límite y entradas incorrectas.

• Comprobar la gestión de ficheros y recursos.

• Verificar que las clases compilan y se ejecutan.

• Comprobar que las modificaciones del inventario afectan únicamente al campo correspondiente.

• Comprobar el caso en que una categoría no tiene registros.

• Comprobar el caso de un fichero con menos de cuatro bytes en la comprobación ZIP.

12. Última petición

Después de revisar y organizar el contenido, el usuario pidió crear un PDF con el chat. Se generó una primera versión y posteriormente solicitó ampliarla. Este documento corresponde a la versión ampliada.

13. Nota sobre los datos no disponibles

Este PDF conserva la información disponible en la conversación y en el documento aportado. No se han inventado los valores de los equipos que deberían proceder de ArraysInventario.txt. Si ese fichero se aporta posteriormente, sus datos pueden incorporarse de forma exacta.